Analysis and Design of Data Structures

Massimo Benerecetti & Fabio Mogavero



containers

special

Dat

Stack

```
Top: Dat
Pop: void
Push: Dat → void
SwapTop: Dat → void
```

Dat

Queue

```
Head: Dat
Dequeue: void
Enqueue: Dat → void
```

linear

Dat

Sequence

```
GetAt: Nat → Dat
SetAt: Dat × Nat → void
Search: Dat → Nat
Exists: Dat → bool
SubSequence: Nat × Nat → Seq
IsEqual: Sequence → bool
```

collections

Dat

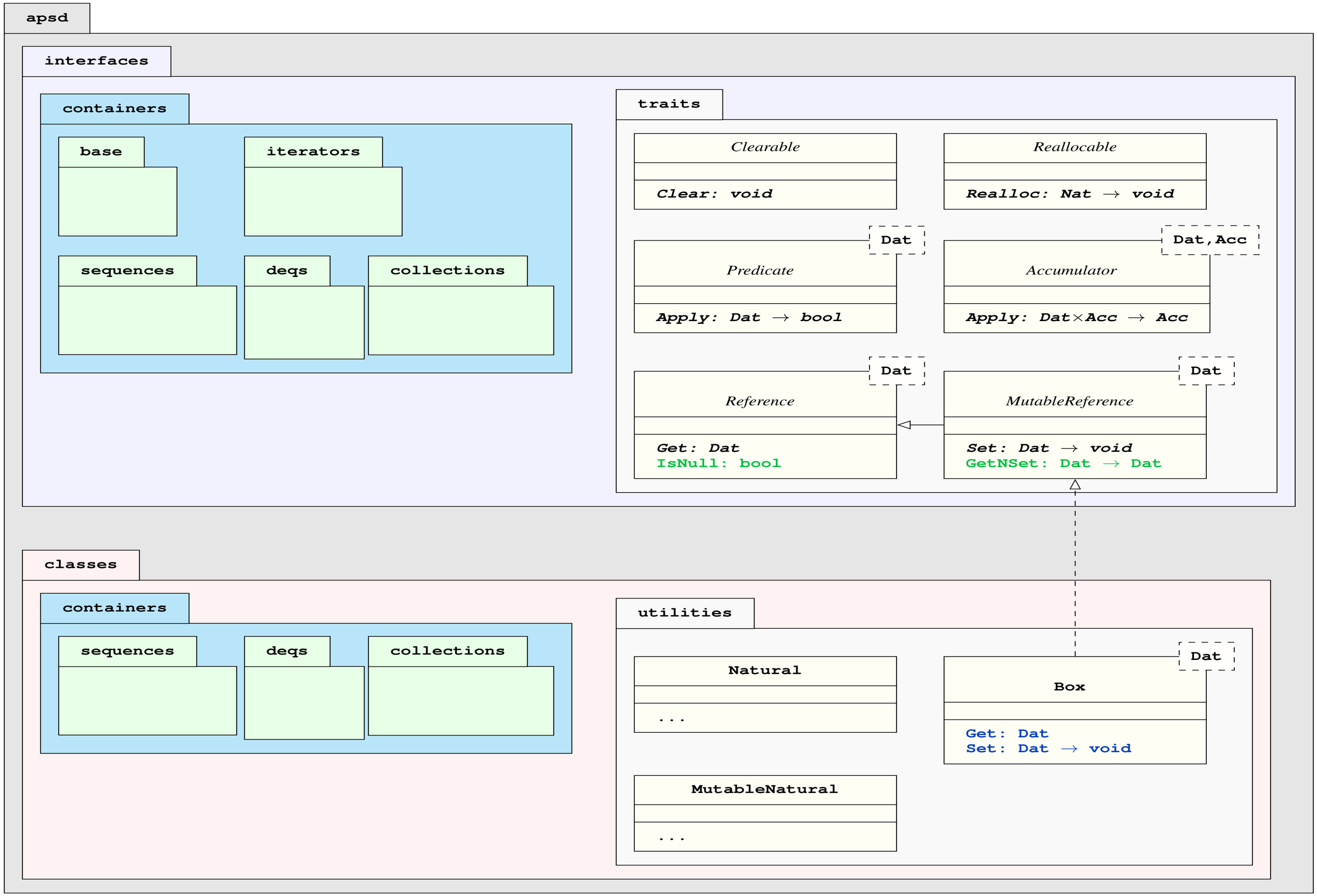
Set

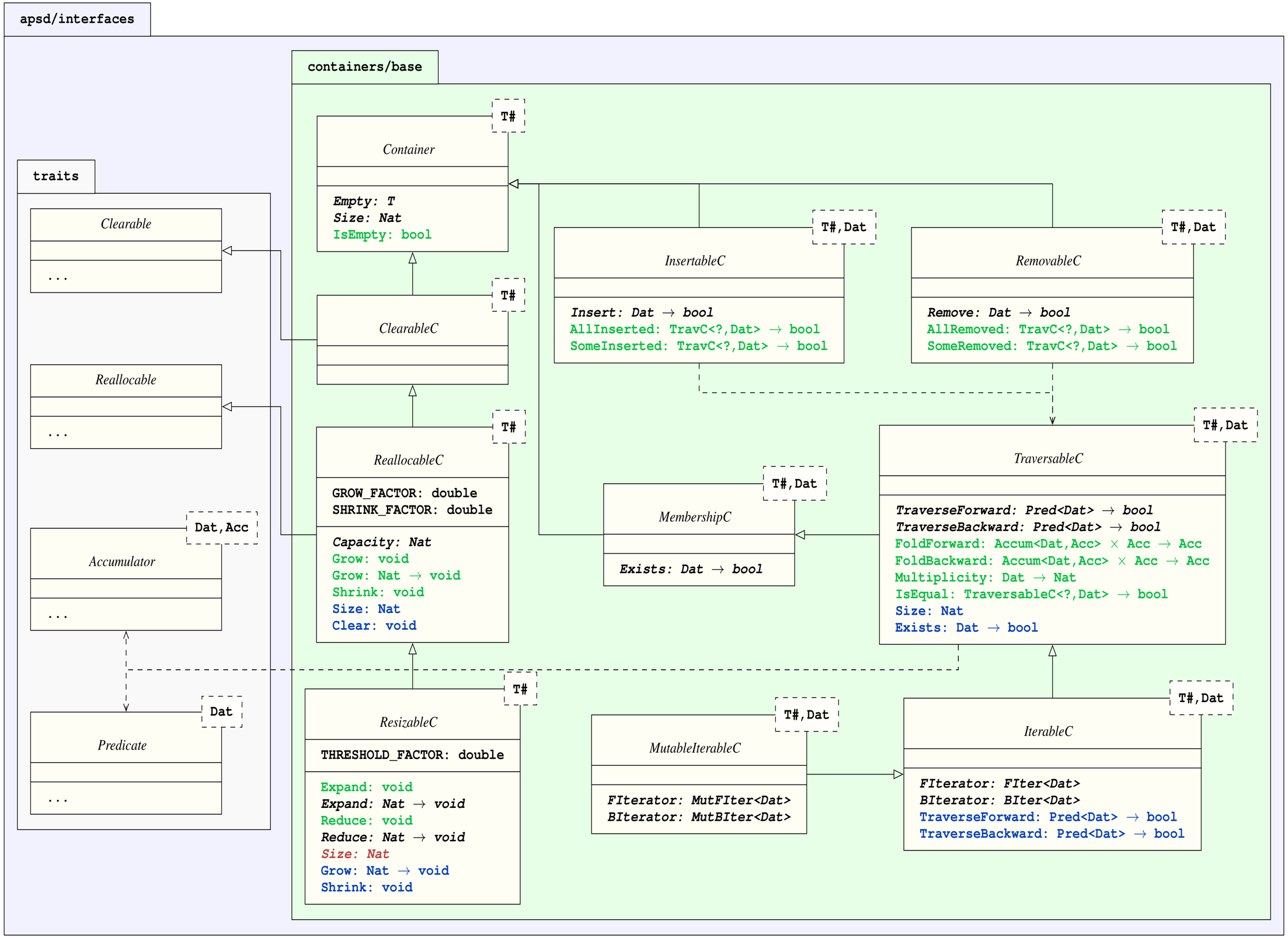
```
Exists: Dat → bool
Insert: Dat → bool
Remove: Dat → bool
Union: Set → void
Difference: Set → void
Intersection: Set → void
IsEqual: Set → bool
```

Dat*

OrderedSet

```
Min/Max: Dat
RemoveMin/Max: void
Predecessor/Successor: Dat → Dat
RemovePredecessor/Successor: Dat → void
```


apsd
interfaces
containers
base
iterators
sequences
deqs
collections
traits
Clearable
Clear: void
Reallocable
Realloc: Nat → void
Predicate
Dat
Apply: Dat → bool
Accumulator
Dat,Acc
Apply: Dat×Acc → Acc
Reference
Dat
Get: Dat
IsNull: bool
MutableReference
Dat
Set: Dat → void
GetNSet: Dat → Dat
classes
containers
sequences
deqs
collections
utilities
Natural
...
MutableNatural
...
Box
Dat
Get: Dat
Set: Dat → void


apsd/interfaces
containers/base
traits
Clearable
...
Reallocable
...
Dat,Acc
Accumulator
...
Dat
Predicate
...
T#
Container
Empty: T
Size: Nat
IsEmpty: bool
T#
ClearableC
T#
ReallocableC
GROW_FACTOR: double
SHRINK_FACTOR: double
Capacity: Nat
Grow: void
Grow: Nat → void
Shrink: void
Size: Nat
Clear: void
T#
ResizableC
THRESHOLD_FACTOR: double
Expand: void
Expand: Nat → void
Reduce: void
Reduce: Nat → void
Size: Nat
Grow: Nat → void
Shrink: void
T#,Dat
InsertableC
Insert: Dat → bool
AllInserted: TravC<?,Dat> → bool
SomeInserted: TravC<?,Dat> → bool
T#,Dat
RemovableC
Remove: Dat → bool
AllRemoved: TravC<?,Dat> → bool
SomeRemoved: TravC<?,Dat> → bool
T#,Dat
MembershipC
Exists: Dat → bool
T#,Dat
MutableIterableC
FIterator: MutFIter<Dat>
BIterator: MutBIter<Dat>
T#,Dat
TraversableC
TraverseForward: Pred<Dat> → bool
TraverseBackward: Pred<Dat> → bool
FoldForward: Accum<Dat,Acc> × Acc → Acc
FoldBackward: Accum<Dat,Acc> × Acc → Acc
Multiplicity: Dat → Nat
IsEqual: TraversableC<?,Dat> → bool
Size: Nat
Exists: Dat → bool
T#,Dat
IterableC
FIterator: FIter<Dat>
BIterator: BIter<Dat>
TraverseForward: Pred<Dat> → bool
TraverseBackward: Pred<Dat> → bool

- `interface TraversableContainer<T,Data>`:
  - `boolean TraverseForward(Predicate<Data> pred)` e `boolean TraverseBackward(Predicate<Data> pred)`: Ciascuna scorre (in forward e in backward, rispettivamente) i dato del contenitore e, su ogni dati, verifica se esso soddisfa il predicato "pred" fornito in ingresso. Termina anticipatamente se trova un dato che soddisfa il predicato

  - `Acc FoldForward(Accumulator<Data, Acc> fun, Acc ini)` e `Acc FoldBackward(Accumulator<Data, Acc> fun, Acc ini)`: Ciascuna sintetizza i dati nel contenitore in un unico oggetto accumulatore "acc" inizializzato al valore "ini", applicando iterativamente la funzione di accumulazione "fun" (in forward e backward, rispettivamente) a ogni dato presente e al valore corrente dell'accumulatore

  - `Natural Multiplicity(Data dat)`: Restituisce il numero di occorrenze di dat all'interno del container

  - `boolean IsEqual(TraversableC<?,Dat> con)`: verifica se il contenitore corrente contiene gli stessi dati, e nella stessa numerosità, del contenitore "con"

- `interface InsertableContainer<T,Data>`:
  - `boolean AllInserted(TraversableContainer<?,Data> con)`: Inserisce gli elementi di "con" nel container corrente, restituendo true se tutti sono stati correttamente inseriti

  - `boolean SomeInserted(TraversableContainer<?,Data> con)`: Inserisce gli elementi di "con" nel container corrente, restituendo true se almeno uno è stato correttamente inserito

- `interface RemovableContainer<T,Data>`:
  - `boolean AllRemoved(TraversableContainer<Data> con)`: Rimuove gli elementi di "con" dal container corrente, restituendo true se tutti sono stati correttamente rimossi

  - `boolean SomeRemoved(TraversableContainer<Data> con)`: Rimuove gli elementi di "con" dal container corrente, restituendo true se almeno uno è stato correttamente rimossi

- `interface ReallocableContainer<T,Data>`:
  - `void Grow(Natural dim)`: Realloca lo spazio per il contenitore in modo che la nuova dimensione sia corrisponda al prodotto di quella corrente più "dim" per `GROW_FACTOR`

  - `void Grow()`: Realloca lo spazio per il contenitore in modo che la nuova dimensione corrisponda al prodotto di quella corrente più 1 per `GROW_FACTOR`

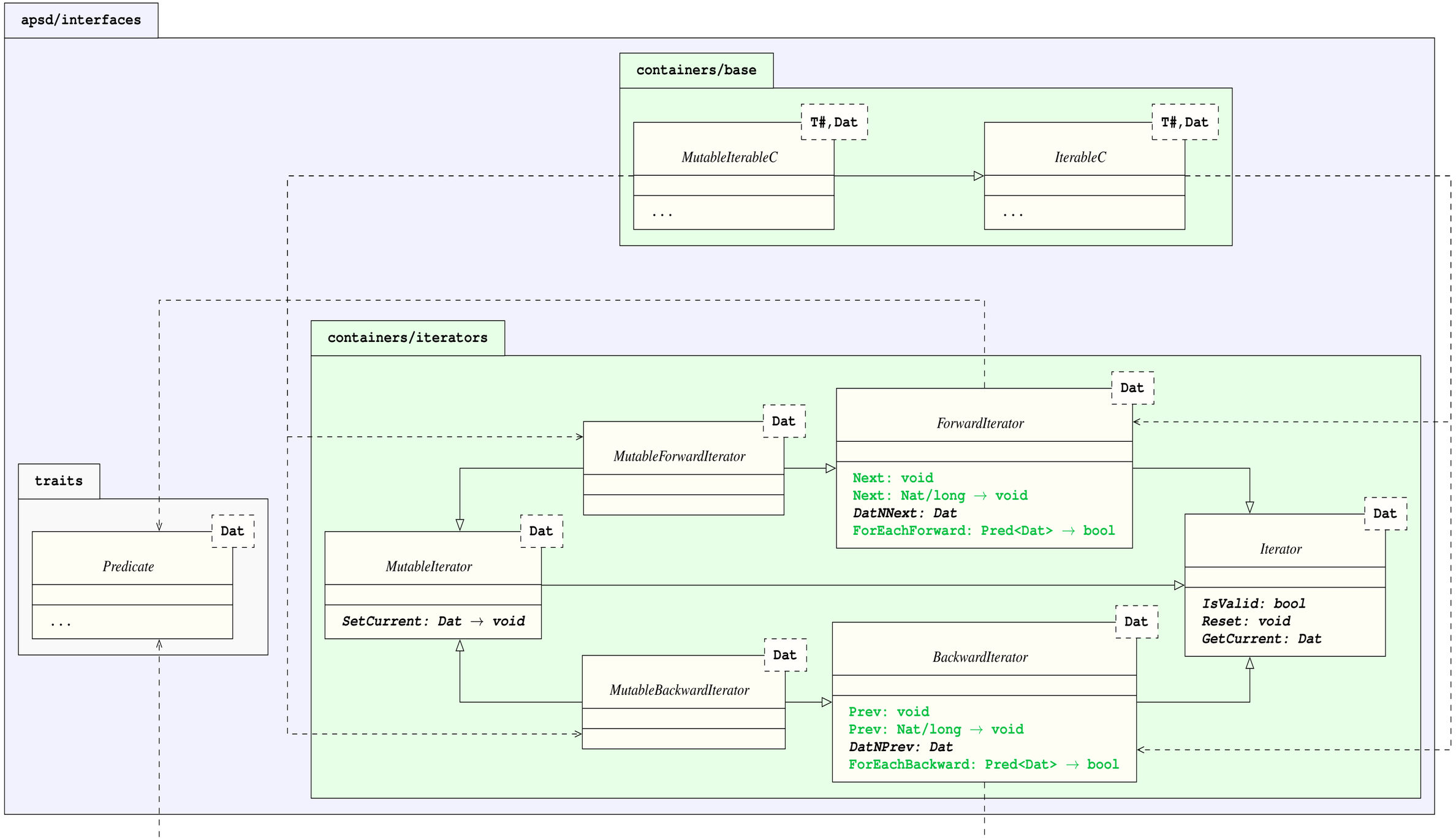
  - `void Shrink()`: Realloca lo spazio per il contenitore in modo che la nuova dimensione corrisponda al prodotto di quella corrente diviso per `SHRINK_FACTOR`

- `interface ResizableContainer<T,Data>`:
  - `void Expand(Natural num)`: Espande la "size" del contenitore di "num" elementi, eventualmente ridimensionandolo

  - `void Expand()`: Espande la "size" del contenitore di un elemento, eventualmente ridimensionandolo

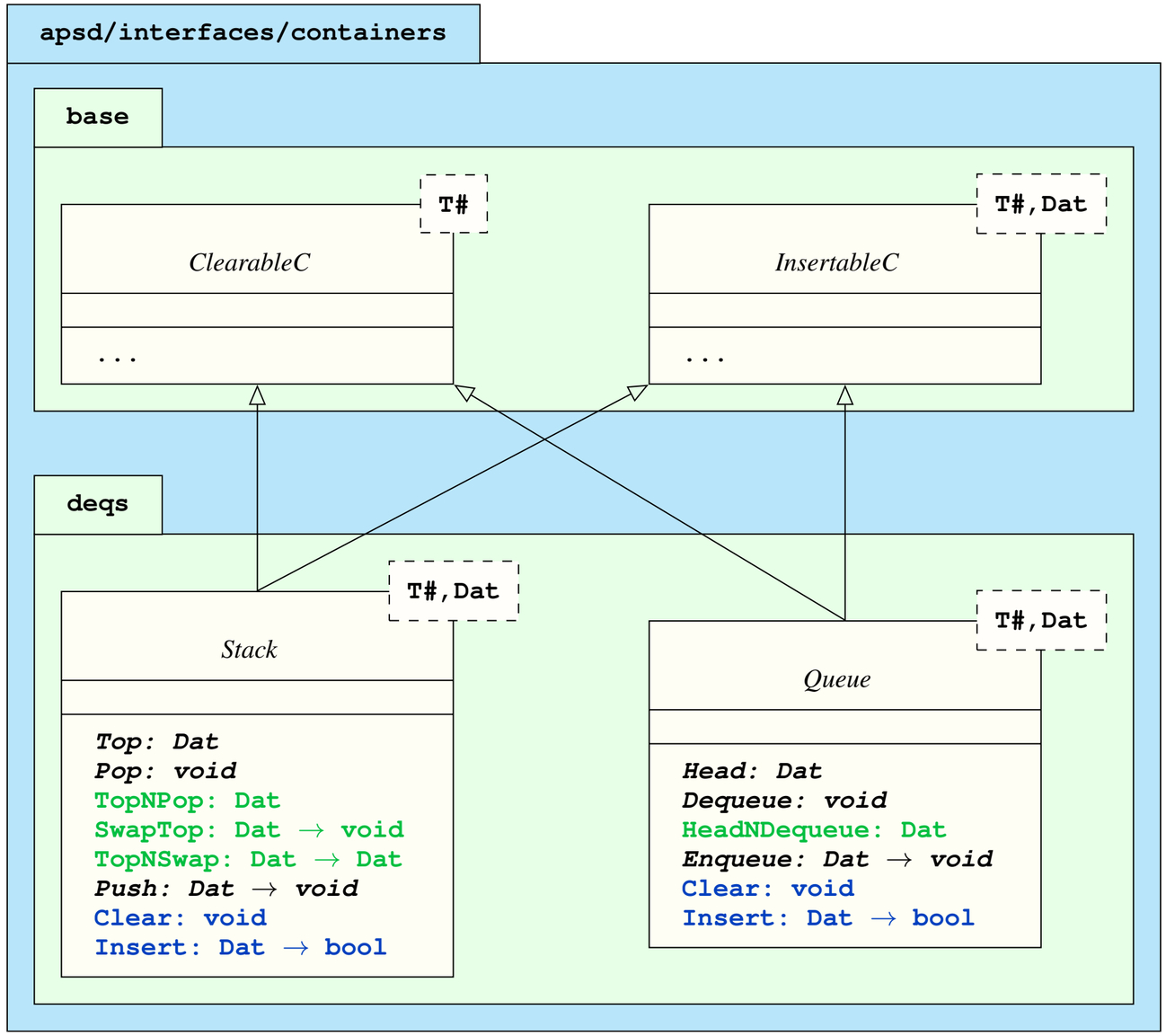
  - `void Reduce(Natural num)`: Riduce la "size" del contenitore di "dim" elementi, eventualmente ridimensionandolo

  - `void Reduce()`: Riduce la "size" del contenitore di un elemento, eventualmente ridimensionandolo



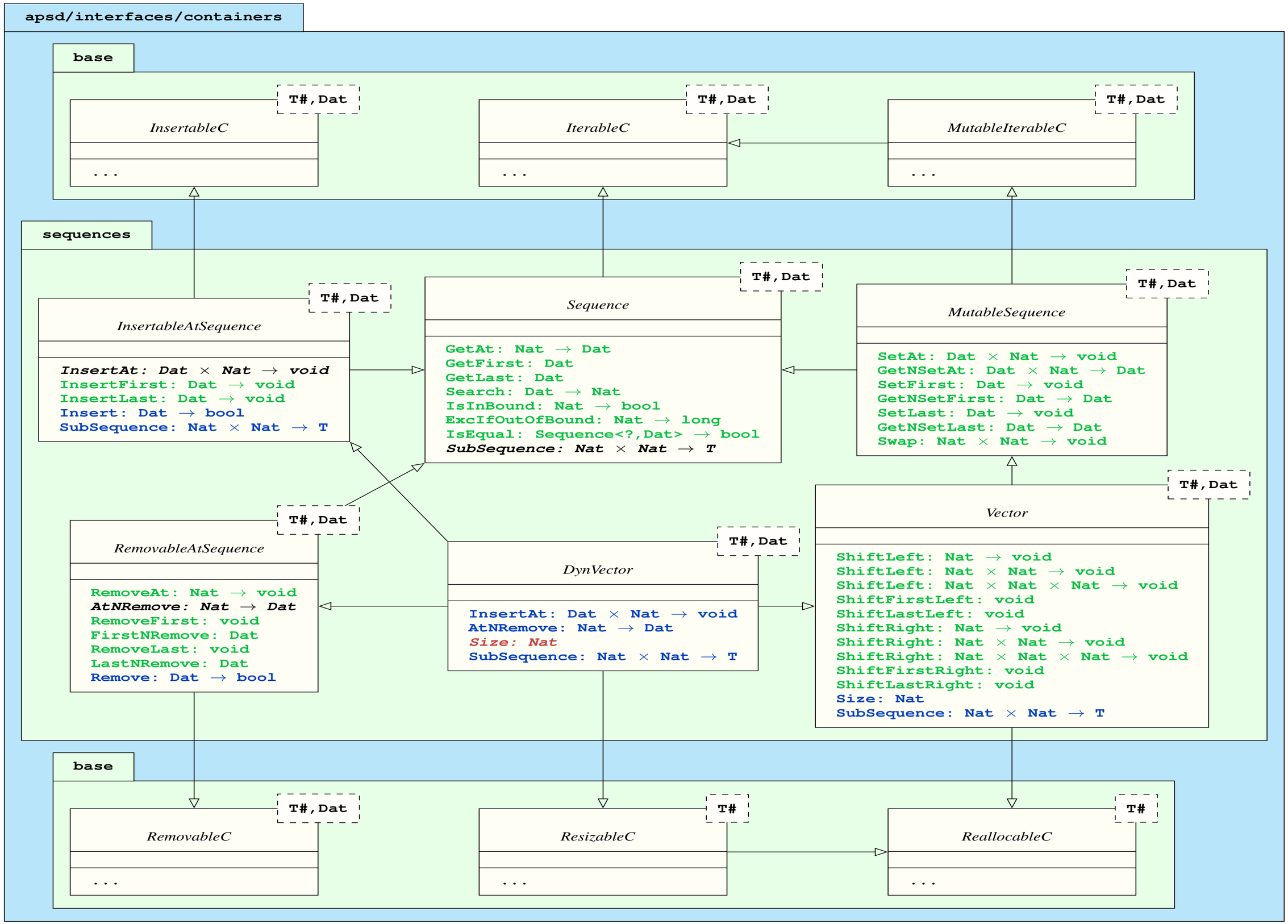
- interface Forward/BackwardIterator<T,Data>:
  - boolean ForEachForward/Backward(Predicate<Data> pred): Applicano a ogni dato accessibile tramite il forward iterator (backward iterator, rispettivamente) il predicato in input "pred" (terminano in anticipo se "pred" restituisce valore true)

  - `Data DatNNext/Prev()`: estraggono e restituiscono il dato attualmente referenziato dall'iteratore e, al contempo, spostano l'iteratore di una posizione in avanti (all'indietro, rispettivamente)



- `interface Stack<T,Data>` :
  - `Data TopNPop()` : Rimuove il primo elemento dello stack e restituisce il dato in esso presente
  - `void SwapTop(Data dat)`: Rimpiazza il dato in cima allo stack col dato fornito in ingresso
  - `Data TopNSwap(Data newdat)`: Rimpiazza il dato in cima allo stack col dato fornito in ingresso e restituisce il dato inizialmente presente in cima

- `interface Queue<T,Data>`:
  - `Data HeadNDequeue()`: Rimuove il dato in testa alla coda e ne restituisce il valore


apsd/interfaces/containers
base
T#,Dat
InsertableC
...
T#,Dat
IterableC
...
T#,Dat
MutableIterableC
...
sequences
T#,Dat
InsertableAtSequence
InsertAt: Dat × Nat → void
InsertFirst: Dat → void
InsertLast: Dat → void
Insert: Dat → bool
SubSequence: Nat × Nat → T
T#,Dat
Sequence
GetAt: Nat → Dat
GetFirst: Dat
GetLast: Dat
Search: Dat → Nat
IsInBound: Nat → bool
ExcIfOutOfBound: Nat → long
IsEqual: Sequence<?,Dat> → bool
SubSequence: Nat × Nat → T
T#,Dat
MutableSequence
SetAt: Dat × Nat → void
GetNSetAt: Dat × Nat → Dat
SetFirst: Dat → void
GetNSetFirst: Dat → Dat
SetLast: Dat → void
GetNSetLast: Dat → Dat
Swap: Nat × Nat → void
T#,Dat
Vector
ShiftLeft: Nat → void
ShiftLeft: Nat × Nat → void
ShiftLeft: Nat × Nat × Nat → void
ShiftFirstLeft: void
ShiftLastLeft: void
ShiftRight: Nat → void
ShiftRight: Nat × Nat → void
ShiftRight: Nat × Nat × Nat → void
ShiftFirstRight: void
ShiftLastRight: void
Size: Nat
SubSequence: Nat × Nat → T
T#,Dat
RemovableAtSequence
RemoveAt: Nat → void
AtNRemove: Nat → Dat
RemoveFirst: void
FirstNRemove: Dat
RemoveLast: void
LastNRemove: Dat
Remove: Dat → bool
T#,Dat
DynVector
InsertAt: Dat × Nat → void
AtNRemove: Nat → Dat
Size: Nat
SubSequence: Nat × Nat → T
base
T#,Dat
RemovableC
...
T#
ResizableC
...
T#
ReallocableC
...

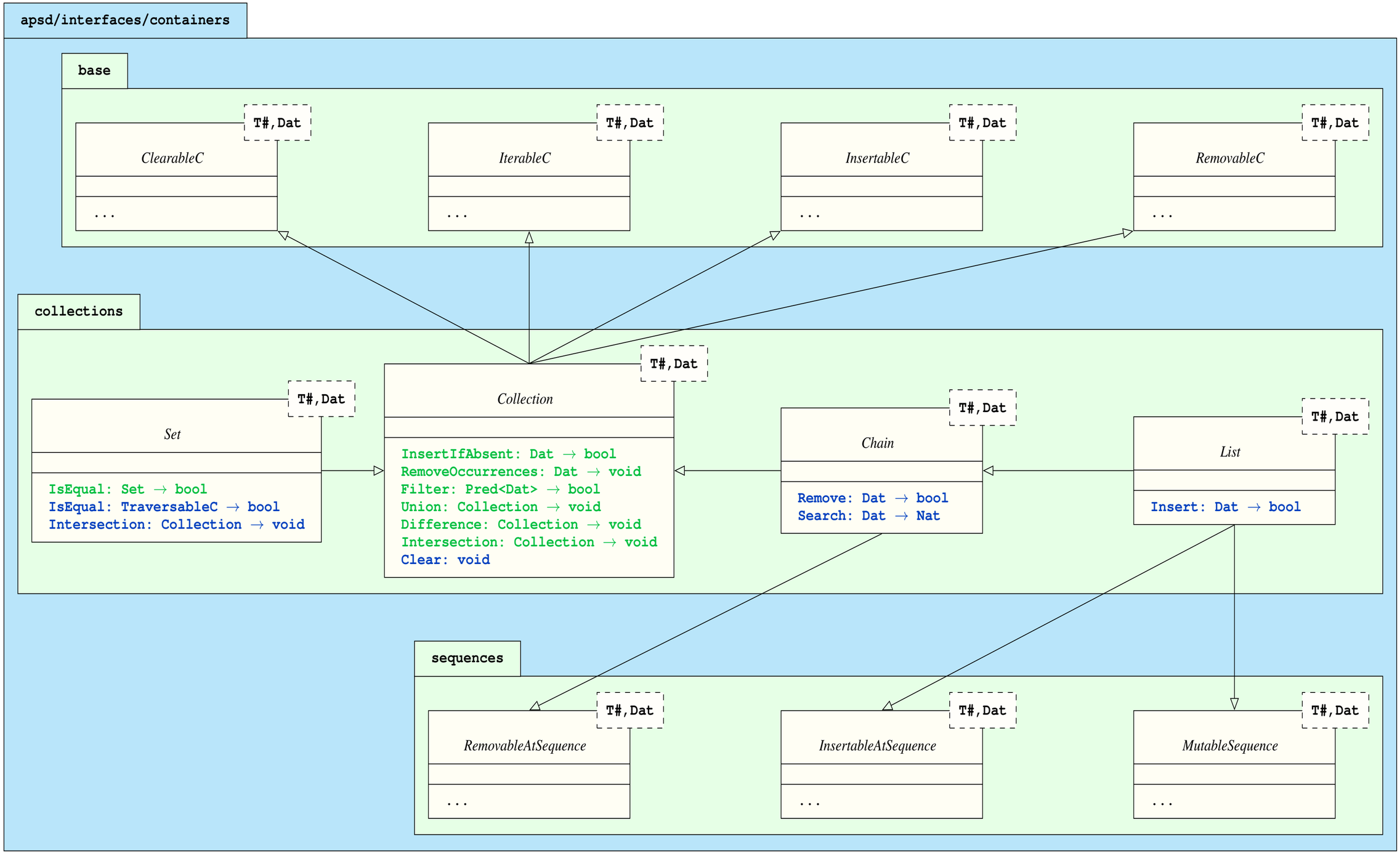
- `interface Sequence<T,Data>`:
  - `boolean IsInBound(Natural num)`: restituisce true se "num" corrisponde a una posizione valida della sequenza
  - `long ExcIfOutOfBound(Natural num)`: restituisce il long contenuto in "num", se corrisponde a una posizione valida della sequenza, e solleva eccezione, altrimenti
  - `boolean IsEqual(Sequence<?, Data> seq)`: Verifica che la sequenza corrente sia identica, stessi dati nelle stesse posizioni, alla sequenza "seq"
  - `T SubSequence(Natural from, Natural to)` : costruisce e restituisce una sequenza con gli elementi contenuti tra le posizioni "from" e "to" nella sequenza corrente

- `interface MutableSequence<T,Data>`:
  - `Data GetNSetAt(Data newdat, Natural pos)`: restituisce il dato in posizione "pos", sostituendolo, al contempo, con il valore "newdat"
  - `Data GetNSetFirst/Last(Data newdat)`: restituisce il dato in prima (rispettivamente, ultima) posizione, sostituendolo, al contempo, con il valore "newdat"
  - `void Swap(Natural fst, Natural snd)`: scambia di posizione i dati nelle posizioni "fst" e "snd"

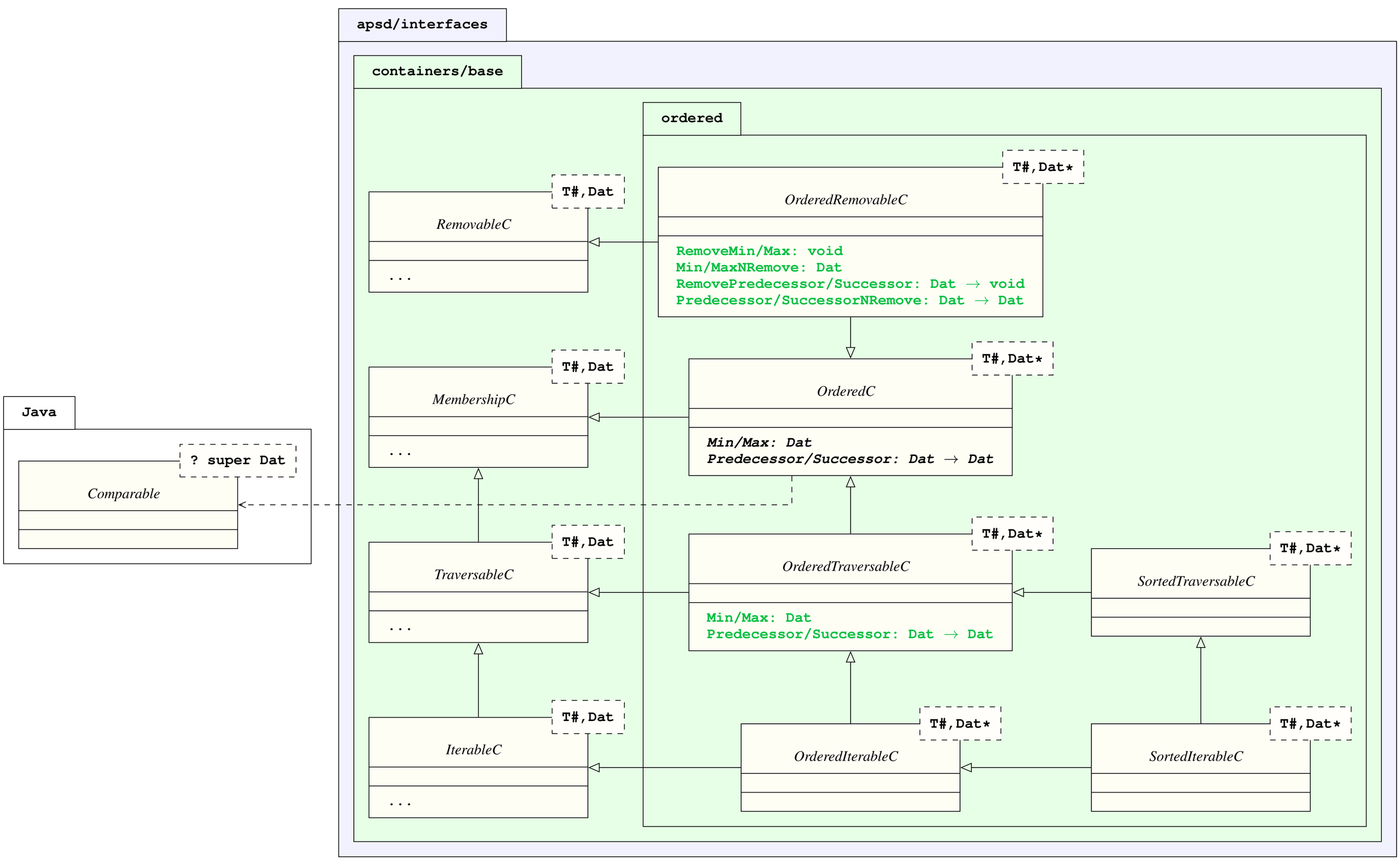
- `interface RemovableAtSequence<T,Data>`:
  - `Data AtNRemove(Natural pos)`: Rimuove il dato in posizione "pos" e lo restituisce
  - `Data First/LastNRemove()`: Rimuove il dato in prima (rispettivamente, ultima) posizione e lo restituisce

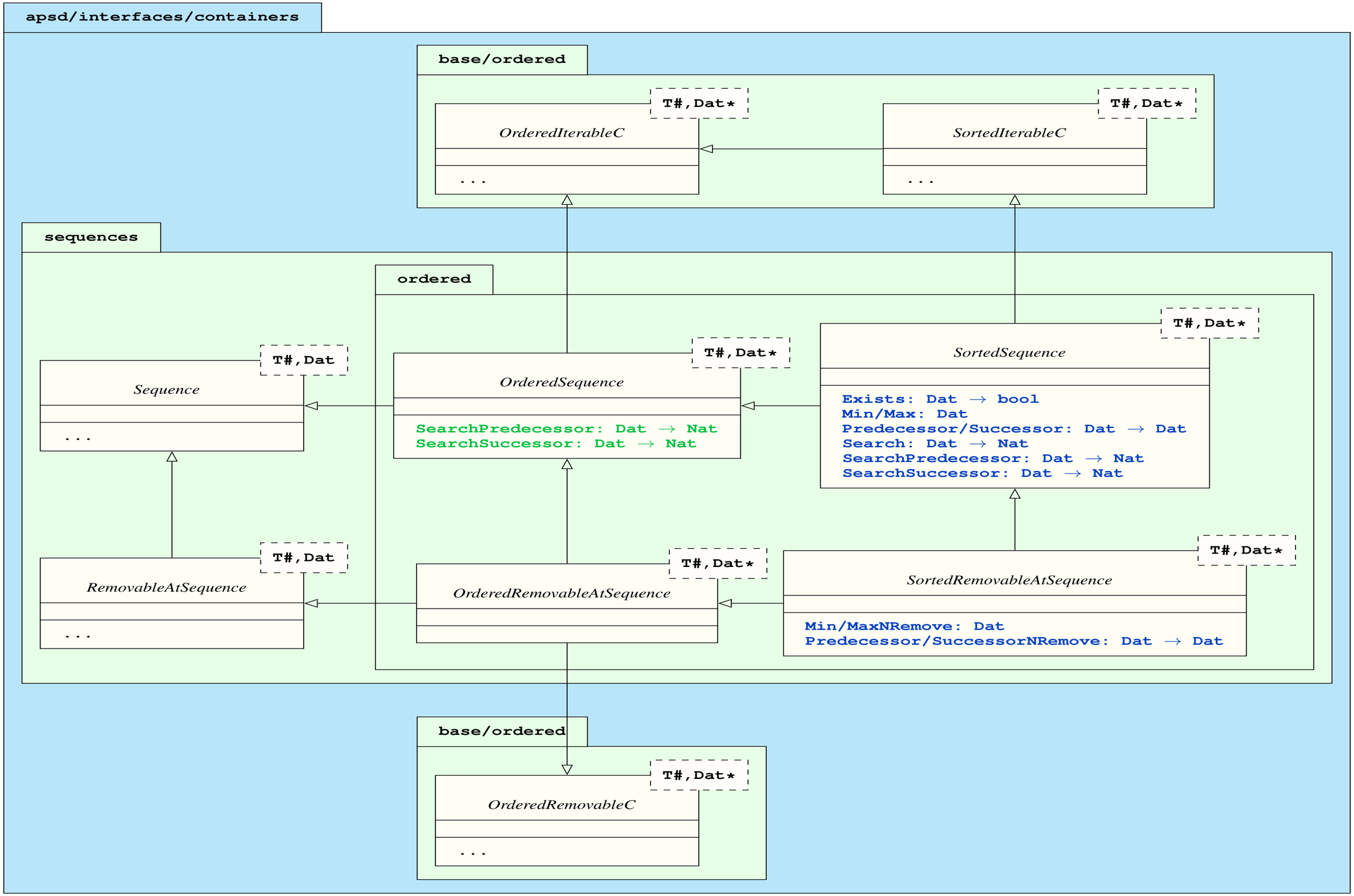
- `interface Vector<T,Data>`:
  - `void ShiftLeft/Right(Natural srt, Natural end, Natural num)`: Sposta a sinistra/destra di "num" posizioni tutti gli elementi contenuti nell'intervallo aperto a destra ["srt", "end"), inserendo il valore "null" al posto dei valori spostati
  - `void ShiftLeft/Right(Natural pos, Natural num)`: Sposta a sinistra (rispettivamente, a destra) di "num" posizioni tutti gli elementi dalla posizione "pos" alla fine
  - `void ShiftLeft/Right(Natural pos)`: Sposta a sinistra (rispettivamente, a destra) di una posizione tutti gli elementi dalla posizione "pos" alla fine
  - `void ShiftFirst/LastRight()` : Sposta a destra di una posizione tutti gli elementi dalla prima (rispettivamente, ultima) posizione alla fine
  - `void ShiftFirst/LastLeft()` : Sposta a sinistra di una posizione tutti gli elementi dalla prima (rispettivamente, ultima) posizione alla fine

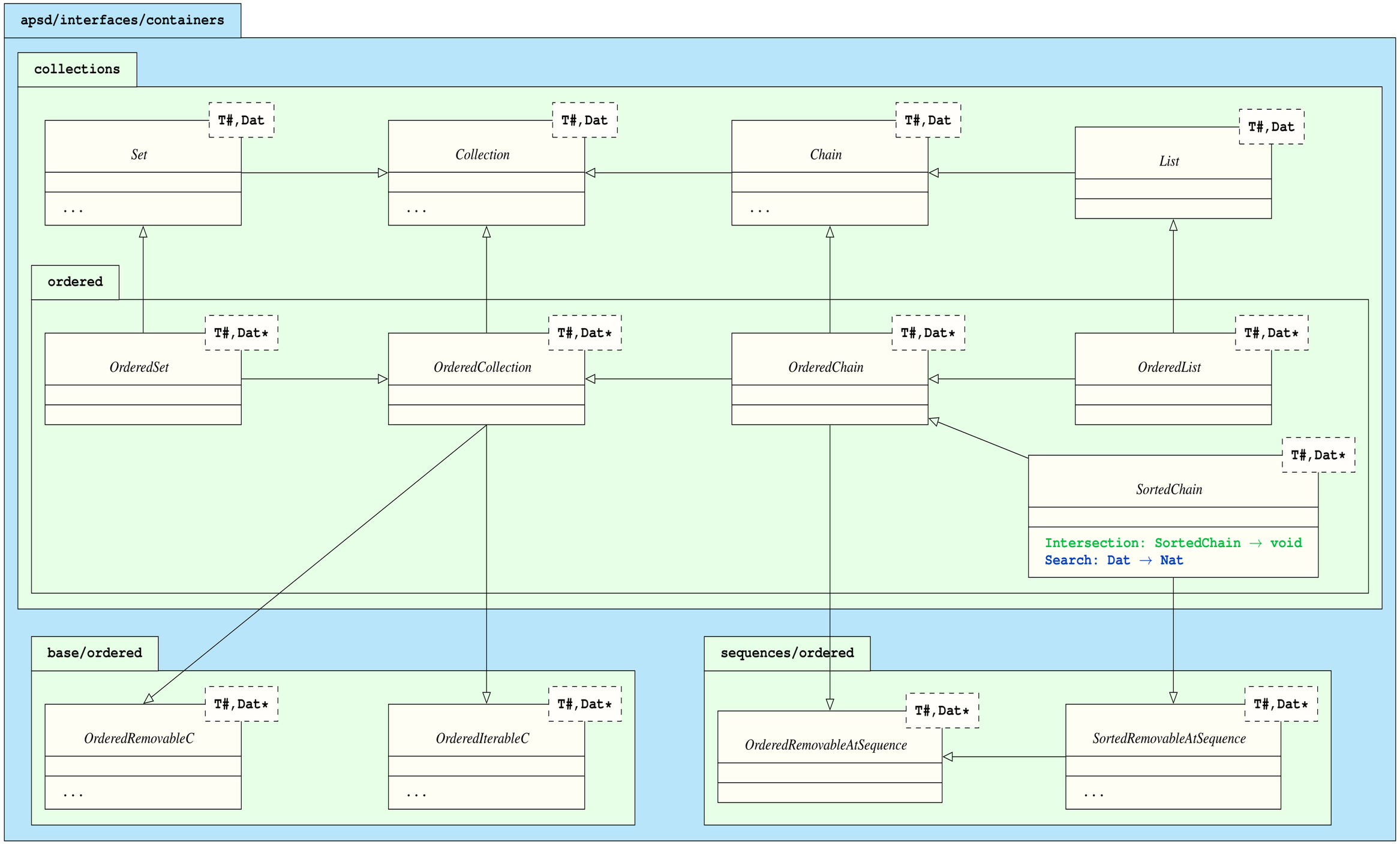
- `interface DynVector<T,Data>`:
  - `Data AtNRemove(Natural pos)`: Rimuove l'elemento in posizione "pos" e ne restituisce il valore

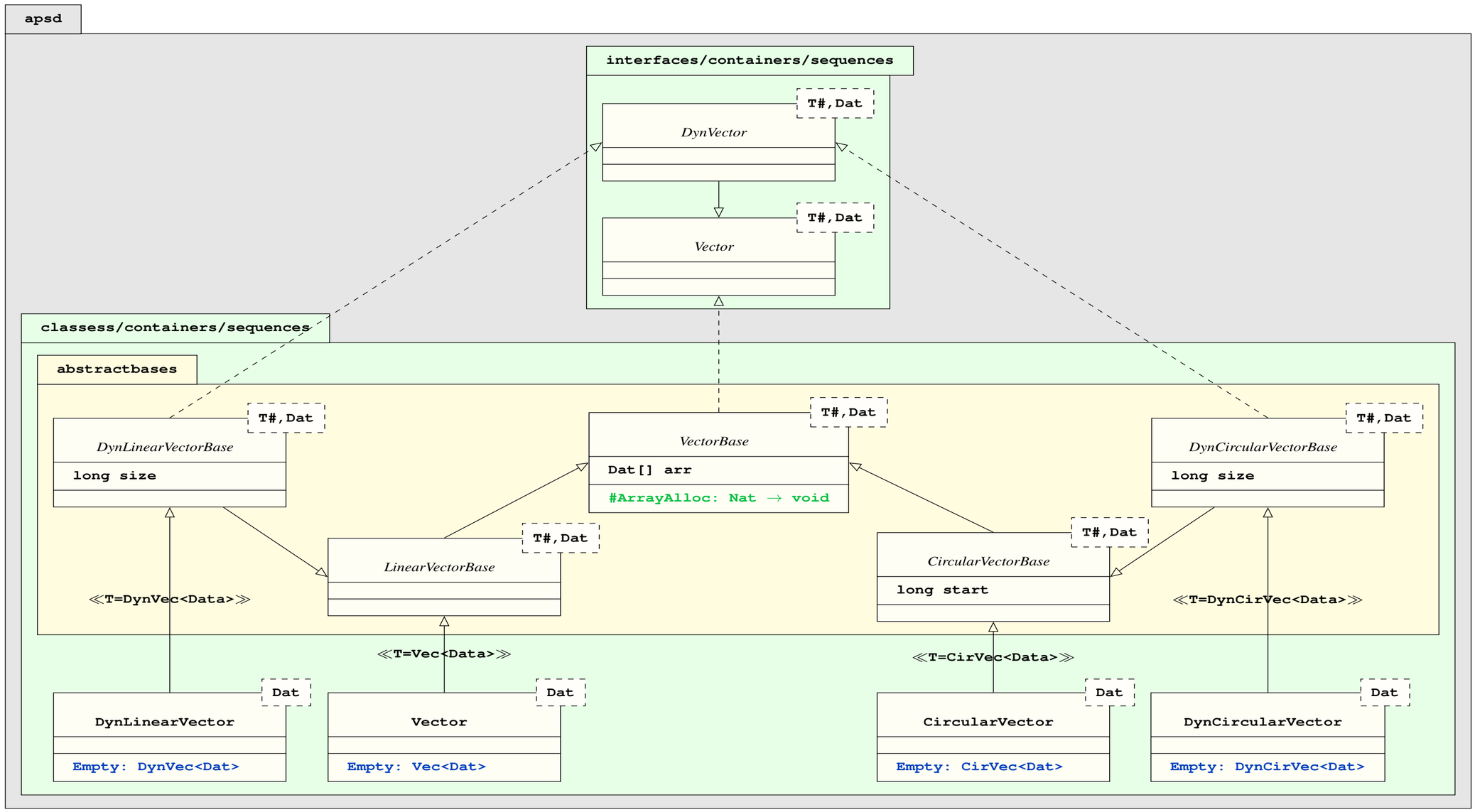

apsd/interfaces/containers
base
ClearableC
T#,Dat
...
IterableC
T#,Dat
...
InsertableC
T#,Dat
...
RemovableC
T#,Dat
...
collections
Set
T#,Dat
IsEqual: Set → bool
IsEqual: TraversableC → bool
Intersection: Collection → void
Collection
T#,Dat
InsertIfAbsent: Dat → bool
RemoveOccurrences: Dat → void
Filter: Pred<Dat> → bool
Union: Collection → void
Difference: Collection → void
Intersection: Collection → void
Clear: void
Chain
T#,Dat
Remove: Dat → bool
Search: Dat → Nat
List
T#,Dat
Insert: Dat → bool
sequences
RemovableAtSequence
T#,Dat
...
InsertableAtSequence
T#,Dat
...
MutableSequence
T#,Dat
...

- `interface Collection<T,Data>`:
  - `boolean InsertIfAbsent(Data dat)`: Inserisce nella chain il dato "dat" se esso non è già presente, restitusce true in caso di inserimento e false altrimenti
  - `RemoveOccurrences(Data dat)`: Rimuove dalla chain tutte le occorrenze del dato "dat"
  - `boolean Filter(Predicate<Data> fun)`: Rimuove tutti dati della collezione che non soddisfano il predicato "fun"
  - `void Union(Collection<?,Data> col)`: Applica sul posto l'operazione di unione della collezione corrente con la collezione "col"
  - `void Difference(Collection<Data> col)`: Applica sul posto l'operazione di differenza alla collezione corrente con la collezione "col"
  - `void Intersection(Collection<?,Data> col)`: Applica sul posto l'operazione di intersezione dell'insieme corrente con la collezione "col"

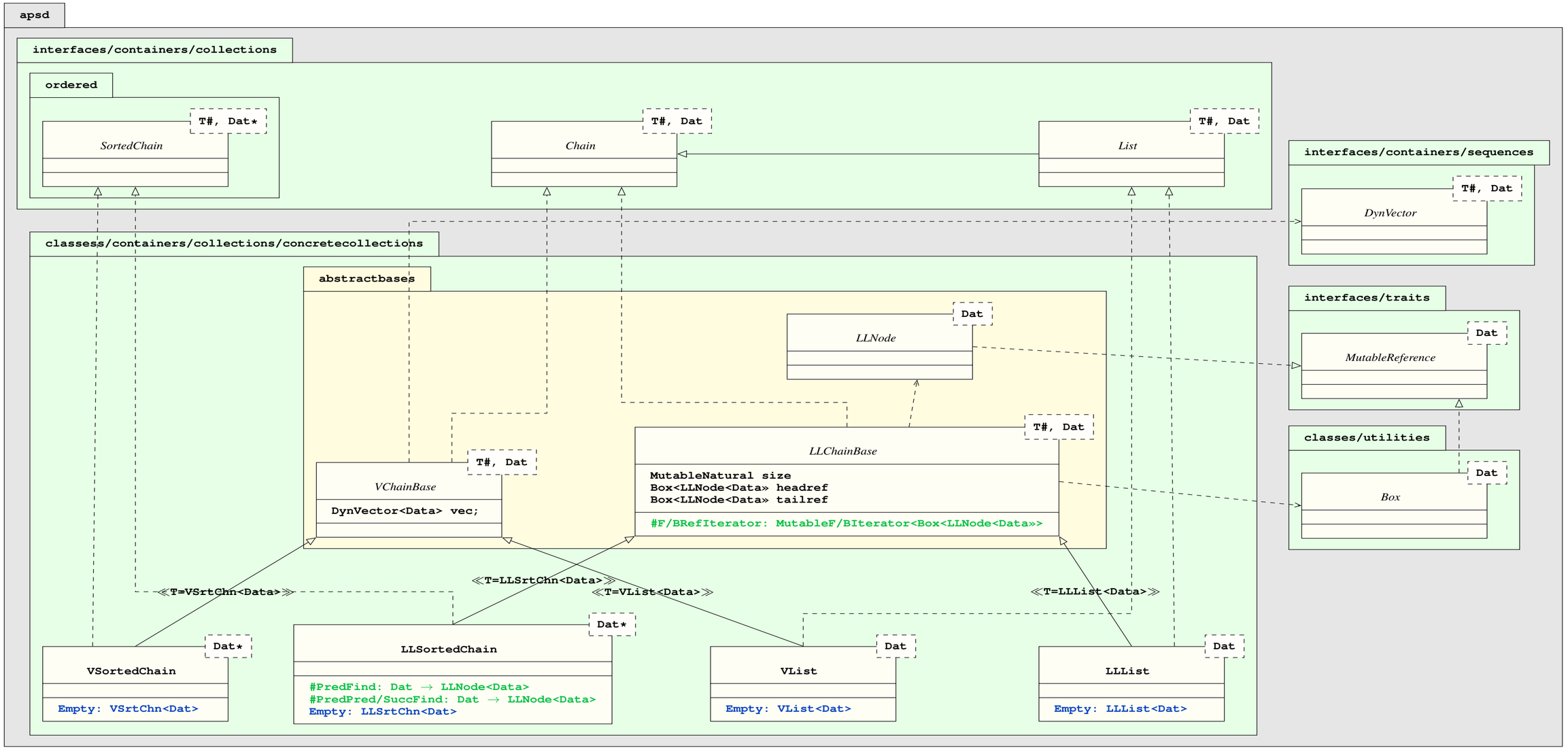

Java
? super Dat
Comparable
apsd/interfaces
containers/base
ordered
T#,Dat
RemovableC
...
T#,Dat*
OrderedRemovableC
RemoveMin/Max: void
Min/MaxNRemove: Dat
RemovePredecessor/Successor: Dat → void
Predecessor/SuccessorNRemove: Dat → Dat
T#,Dat
MembershipC
...
T#,Dat*
OrderedC
Min/Max: Dat
Predecessor/Successor: Dat → Dat
T#,Dat
TraversableC
...
T#,Dat*
OrderedTraversableC
Min/Max: Dat
Predecessor/Successor: Dat → Dat
T#,Dat*
SortedTraversableC
T#,Dat
IterableC
...
T#,Dat*
OrderedIterableC
T#,Dat*
SortedIterableC


apsd/interfaces/containers
base/ordered
T#,Dat*
OrderedIterableC
...
T#,Dat*
SortedIterableC
...
sequences
ordered
T#,Dat
Sequence
...
T#,Dat*
OrderedSequence
SearchPredecessor: Dat → Nat
SearchSuccessor: Dat → Nat
T#,Dat*
SortedSequence
Exists: Dat → bool
Min/Max: Dat
Predecessor/Successor: Dat → Dat
Search: Dat → Nat
SearchPredecessor: Dat → Nat
SearchSuccessor: Dat → Nat
T#,Dat
RemovableAtSequence
...
T#,Dat*
OrderedRemovableAtSequence
T#,Dat*
SortedRemovableAtSequence
Min/MaxNRemove: Dat
Predecessor/SuccessorNRemove: Dat → Dat
base/ordered
T#,Dat*
OrderedRemovableC
...


apsd/interfaces/containers
collections
Set
T#,Dat
...
Collection
T#,Dat
...
Chain
T#,Dat
...
List
T#,Dat
ordered
OrderedSet
T#,Dat*
OrderedCollection
T#,Dat*
OrderedChain
T#,Dat*
OrderedList
T#,Dat*
SortedChain
T#,Dat*
Intersection: SortedChain → void
Search: Dat → Nat
base/ordered
OrderedRemovableC
T#,Dat*
...
OrderedIterableC
T#,Dat*
...
sequences/ordered
OrderedRemovableAtSequence
T#,Dat*
SortedRemovableAtSequence
T#,Dat*
...

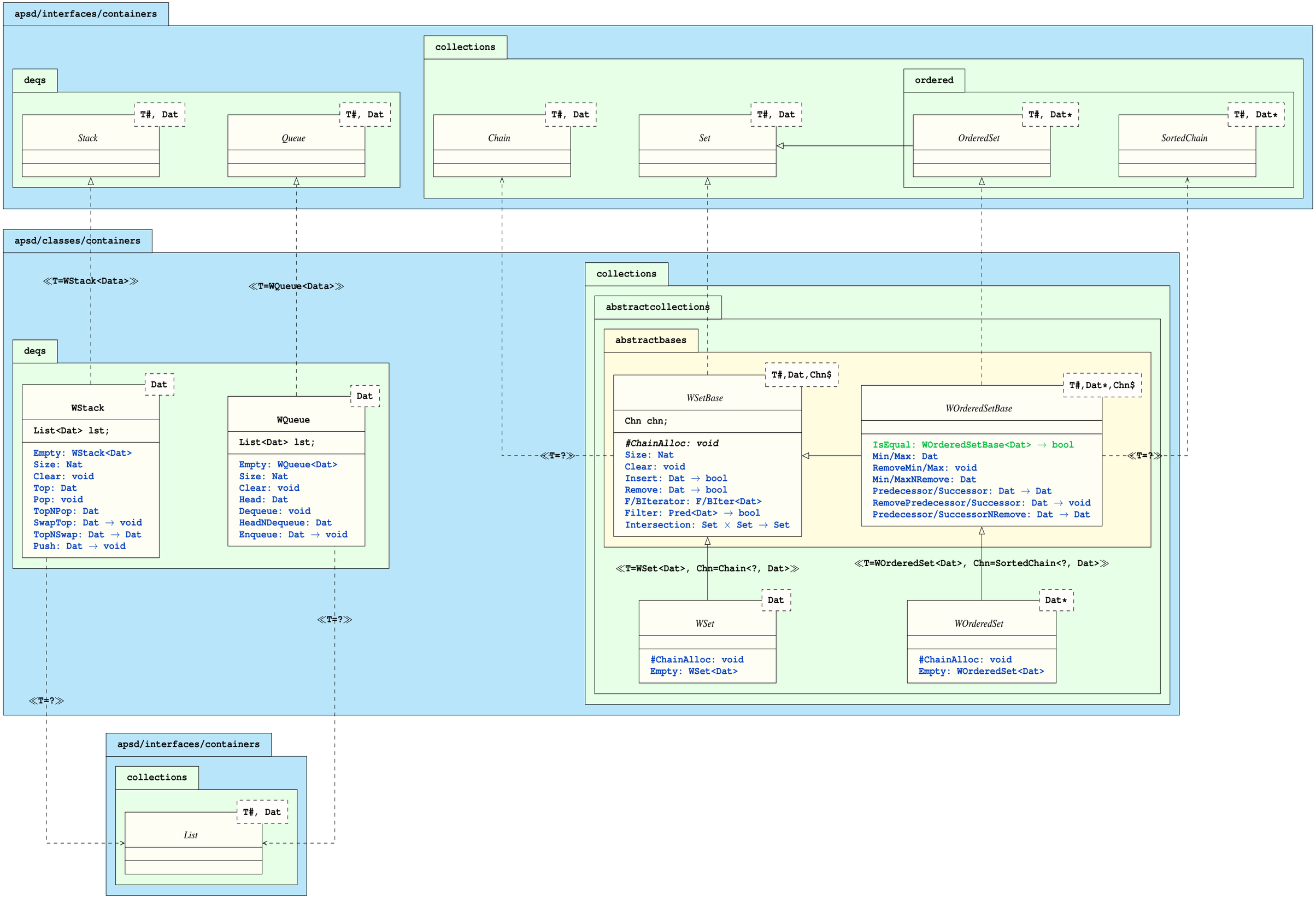


Methods	VecB	LinVecB	CirVecB	DLinVecB	DCirVecB
Size : Nat				✓	✓
Clear : void	✓			✓	✓
Realloc : Nat → void		✓	✓	✓	✓
Capacity : Nat	✓				
Expand : Nat → void				✓	✓
Reduce : Nat → void				✓	✓
F/BIterator : MutableForward/BackwardIterator	✓				
GetAt : Nat → Dat	×	✓	✓		
SetAt : Dat × Nat → void	×	✓	✓		
ShiftLeft/Right : Nat × Nat → void			✓		
ArrayAlloc : Nat → void	✓		✓	✓	✓



Methods	VChainB	LLChainB
Size : Nat	✓	✓
Clear : void	✓	✓
Remove : Dat → bool		✓
F/BIterator : Forward/BackwardIterator	✓	✓
GetAt : Nat → Dat	✓	
GetFirst/Last : Dat		✓
SubSequence : Nat × Nat → Sequence	✓	✓
AtNRemove : Nat → Dat	✓	✓
RemoveFirst/Last : void		✓
First/LastNRemove : Dat		✓
Filter : Pred<Dat> → bool	✓	✓

Methods	VSChain	LLSChain	VList	LLList
F/BIterator : MutableForward/BackwardIterator			✓	✓
Insert : Dat → bool	✓	✓		
Remove : Dat → bool		✓		
Search : Dat → Nat		✓		
SearchPredecessor/Successor : Dat → Nat		✓		
SetAt : Dat × Nat → void			✓	✓
SetFirst/Last : Dat → void				✓
InsertAt : Dat × Nat → void			✓	✓
InsertFirst/Last : Dat → void				✓
The six methods for Predecessor and Successor		✓		
InsertIfAbsent : Dat → bool	✓	✓		
RemoveOccurrences : Dat → void	✓	✓		


apsd/interfaces/containers
collections
deqs
T#, Dat
Stack
T#, Dat
Queue
T#, Dat
Chain
T#, Dat
Set
ordered
T#, Dat*
OrderedSet
T#, Dat*
SortedChain
apsd/classes/containers
≪T=WStack<Data>≫
≪T=WQueue<Data>≫
collections
abstractcollections
abstractbases
deqs
Dat
WStack
List<Dat> lst;
Empty: WStack<Dat>
Size: Nat
Clear: void
Top: Dat
Pop: void
TopNPop: Dat
SwapTop: Dat → void
TopNSwap: Dat → Dat
Push: Dat → void
Dat
WQueue
List<Dat> lst;
Empty: WQueue<Dat>
Size: Nat
Clear: void
Head: Dat
Dequeue: void
HeadNDequeue: Dat
Enqueue: Dat → void
≪T=?≫
≪T=?≫
≪T=?≫
≪T=?≫
T#,Dat,Chn$
WSetBase
Chn chn;
#ChainAlloc: void
Size: Nat
Clear: void
Insert: Dat → bool
Remove: Dat → bool
F/BIterator: F/BIter<Dat>
Filter: Pred<Dat> → bool
Intersection: Set × Set → Set
T#,Dat*,Chn$
WOrderedSetBase
IsEqual: WOrderedSetBase<Dat> → bool
Min/Max: Dat
RemoveMin/Max: void
Min/MaxNRemove: Dat
Predecessor/Successor: Dat → Dat
RemovePredecessor/Successor: Dat → void
Predecessor/SuccessorNRemove: Dat → Dat
≪T=WSet<Dat>, Chn=Chain<?, Dat>≫
≪T=WOrderedSet<Dat>, Chn=SortedChain<?, Dat>≫
Dat
WSet
#ChainAlloc: void
Empty: WSet<Dat>
Dat*
WOrderedSet
#ChainAlloc: void
Empty: WOrderedSet<Dat>
apsd/interfaces/containers
collections
T#, Dat
List

- `class VectorBase<T,Data>`:
  - `protected void ArrayAlloc(Natural dim)`: Crea un array Java della dimensione richiesta "dim" e lo assegna al campo "arr" che cosrtituisce l'implementazione dell'oggetto vettore corrente (usata dai costruttori di vettori concreti).

- `class LLChainBase<T,Data>`:
  - `protected ForwardIterator<Box<LLNode<Data>>> FRefIterator()`: Restituisce un iteratore che scorre i nodi (non i dati) della lista in ordine forward
  - `protected BackwardIterator<Box<LLNode<Data>>> BRefIterator()`: Restituisce un iteratore che scorre i nodi (non i dati) della lista in backward

- `class LLSortedChain<T,Data>`:
  - `protected LLNode<Data> PredFind(Data dat)`: Restituisce il nodo che contiene il predecessore di "dat" nella lista (utilizzato per rimozione di dati)
  - `protected LLNode<Data> PredPredFind(Data dat)`: Restituisce il nodo che contiene il predecessore del predecessore di "dat" nella lista (utilizzato per rimozione di predecessori di dati)
  - `protected LLNode<Data> PredSuccFind(Data dat)`: Restituisce il nodo che contiene il predecessore del successore di "dat" nella lista (utilizzato per rimozione di successori di dati)

- `class WSetBase<T,Data>`:
  - `protected void ChainAlloc()`: Crea una Chain concreta vuota e la assegna al campo "chn" che cosrtituisce l'implementazione dell'oggetto WSet corrente (usata dai costruttori di insiemi concreti).